

Comparative Analysis of Vision-Language-Action Architectures

Pi0 (Flow Matching) vs. Pi0-FAST (Autoregressive Tokenization)
for Language-Conditioned Robotic Manipulation

Kartik Reddy Katam¹ and Gaurav Dharmadhikari¹

¹San José State University, CMPE 188 – Machine Learning
kartikreddy.katam@gmail.com, gaurav.dharmadhikari@sjsu.edu
SJSU IDs: 015765542, 017038177
GitHub: <https://github.com/KartikKatam/libero-vla-analysis>

May 19, 2026

1 Project Goal and Problem Statement

This project investigates a fundamental architectural question in robot learning: how does the choice of action generation mechanism affect the real-world performance of Vision-Language-Action (VLA) models? We compare two state-of-the-art VLA systems — **Pi0** and **Pi0-FAST** — on identical manipulation tasks to understand the practical trade-offs between their respective designs.

Pi0 generates robot actions using *flow matching*, a continuous trajectory generation approach that models actions as samples from a learned distribution. **Pi0-FAST**, by contrast, tokenizes actions into discrete chunks and generates them autoregressively, similar to how large language models generate text. Both models share the same vision and language backbone (PaliGemma-3B: a SigLIP vision encoder + Gemma 2B language model), but diverge sharply in how they produce robot actions.

Our central questions are:

- How do these architectural differences in action generation affect manipulation performance and success rate?
- How do they compare in terms of trajectory quality and smoothness?
- What are the inference speed trade-offs between the two approaches?
- How well does each architecture generalize across task types (spatial reasoning, object identity, goal inference, long-horizon)?

Answering these questions has direct implications for practitioners deploying robotic systems, where choices between model families involve real trade-offs in speed, accuracy, and compute cost.

The headline finding of this pilot is that aggregate success rates are *identical* across the two architectures at 43.6%, but this aggregate hides a structured per-suite divergence: Pi0 wins decisively on spatial reasoning and goal-conditioned tasks, while Pi0-FAST wins decisively on language-to-object grounding. The architectures are **complementary, not strictly ordered** — the right choice depends on which bottleneck the target task imposes.

2 Approach and Implementation

2.1 Pi0: Flow Matching Action Head

Pi0 frames action generation as a flow matching problem [6]. Rather than the stochastic multi-step reverse process of diffusion, flow matching trains a network to regress a velocity field along a straight-line path between noise and data:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_0, x_1} \left[\|v_\theta(x_t, t) - u_t(x_t | x_1)\|^2 \right] \quad (1)$$

where the noised sample lies on the linear interpolation

$$x_t = (1 - t) \cdot x_0 + t \cdot x_1, \quad t \in [0, 1] \quad (2)$$

and the target velocity is

$$u_t(x_t | x_1) = x_1 - x_0. \quad (3)$$

This is a stable MSE objective with no noise-schedule tuning, no ELBO derivation, and no learned reverse-time variance — the target velocity is constant along the entire probability path, which permits aggressive step-count reduction at sampling time.

At inference, **10 Euler integration steps** transform Gaussian noise into a complete action chunk of shape $[H=50, D=32]$, with all 50 actions produced in parallel within each step. The prefix (image + language tokens) is computed once and its KV cache is reused across all 10 flow steps.

Pi0 uses a **separate 311M-parameter action expert** (a narrower Gemma stack with width 1024, vs the 2048-wide backbone) that attends jointly with the PaliGemma-3B VLM backbone through a fused multi-expert attention operation. Total parameters: ~ 3.3 B. The action expert is trained from scratch while the VLM backbone is LoRA-adapted. The separation is motivated by modality mismatch (continuous action vectors have different statistics than language tokens), inference efficiency (only the narrower expert runs per flow step), and pretraining preservation (the VLM’s language priors are minimally perturbed).

The training timestep is sampled from $\tau \sim 0.001 + 0.999 \cdot \text{Beta}(1.5, 1)$, biased toward high-noise regions where the velocity field is hardest to learn and where the first Euler step lands at inference. Attention follows a block-causal pattern: image+language attends to itself bidirectionally; state token attends to image+language+self; action tokens attend to everything but only the action block is causal.

2.2 Pi0-FAST: Autoregressive Action Tokenization

Pi0-FAST applies **FAST** (Frequency-space Action Sequence Tokenization) [2] to convert continuous action chunks into a compact discrete vocabulary, then generates them autoregressively using the same PaliGemma transformer backbone — **no separate action expert, no flow integration**.

The tokenization pipeline processes a raw action chunk through four stages:

1. **Quantile normalization** (1st/99th percentile per dimension) mapping actions to $[-1, 1]$.
2. **Discrete Cosine Transform** (DCT-II) per dimension along the time axis:

$$C_k^d = \sum_{n=0}^{H-1} A_n^d \cos \left[\frac{\pi}{H} \left(n + \frac{1}{2} \right) k \right], \quad k = 0, \dots, H-1 \quad (4)$$

concentrating energy in low-frequency basis functions.

3. **Scale-and-round quantization** with $\gamma = 10$: $\bar{C}_k^d = \text{round}(\gamma \cdot C_k^d)$, producing a sparse integer matrix dominated by zeros at high frequencies.

4. **Byte-Pair Encoding** trained on the flattened integer streams, producing a vocabulary of 1024 IDs.

The DCT step is critical: naive per-step binning produces correlated tokens that the model learns to copy. DCT concentrates smooth trajectories into a few low-frequency coefficients — the same principle JPEG uses on images — achieving up to **13.2× compression** versus naive binning at 50 Hz / 14-DoF control (FAST paper Table I). For LIBERO’s 7-DoF 20 Hz setting, FAST emits ~ 30 tokens per chunk.

A subtle architectural detail: FAST does **not** expand PaliGemma’s embedding matrix. The 1024 FAST IDs **overwrite the least-used tail** of PaliGemma’s 257,152-entry SentencePiece tokenizer (after skipping the last 128 special tokens). This is what allows Pi0-FAST to start directly from the pretrained PaliGemma checkpoint with no embedding re-initialization.

Action tokens are appended to the text vocabulary and the model performs standard next-token prediction with cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|M|} \sum_{t \in M} \log p_{\theta}(y_t \mid y_{<t}, \text{image}, \text{text}, \text{state}) \quad (5)$$

where M is the action-token range. The state is also discretized (256 bins) and placed in the prefix, since there is no action expert to consume continuous proprioception.

2.3 Architectural Comparison

Table 1: Architectural comparison of Pi0 and Pi0-FAST.

Dimension	Pi0 (flow matching)	Pi0-FAST (autoregressive)
Action representation	Continuous chunks (50×32)	DCT-compressed discrete tokens (~ 30 IDs)
Generation paradigm	Velocity-field ODE integration	Next-token prediction (causal)
Loss	MSE on velocity field	Cross-entropy on tokens
Inference cost per chunk	10 forward passes (50 actions batched)	1 prefix prefill + ~ 30 –60 serial decodes
Inference latency on 4090	~ 100 ms/chunk (faster)	~ 750 ms/chunk ($\sim 7\times$ slower)
Training compute to convergence	Standard	$\sim 5\times$ faster than Pi0 (FAST paper §6)
Action expert	Separate 311M-param Gemma module	None — uses backbone directly
Native action-distribution multimodality	Yes (flow matching preserves modes)	Greedy decode collapses modes
Calibrated per-token uncertainty	No closed-form	Yes (softmax over 1024-way vocab)
Quantization error	None (continuous)	≈ 0.1 normalized units round-trip ($\gamma=10$)

A note on inference latency that diverges from the intuitive “fewer steps = faster” framing: **Pi0-FAST is actually slower per chunk** because its 30–60 token decodes cannot be parallelized along time, whereas Pi0’s 10 Euler steps each operate on the full 50-action chunk in parallel. What FAST saves is **training compute**, not inference compute.

2.4 Fine-Tuning Strategy (as executed)

Both models were fine-tuned from public pretrained base checkpoints (`gs://openpi-assets/checkpoints/pi0_base` and `gs://openpi-assets/checkpoints/pi0_fast_base`) using the openpi **LoRA recipes**:

Training configuration: 5,000 LoRA steps, batch size 32, seed 42, cosine learning-rate decay. *Both models reached step 5,000 successfully, but the final-step checkpoint save aborted mid-write due to a disk*

Table 2: LoRA configuration as executed in this pilot.

Component	Pi0 LoRA targets	Pi0-FAST LoRA targets
PaliGemma 2B backbone	attn QKV + FFN, rank 16, $\alpha = 16$	same, rank 16, $\alpha = 16$
Action expert (Gemma 300M)	attn QKV + FFN, rank 32, $\alpha = 32$	N/A (no action expert)
State / action projection heads	dense (not LoRA-wrapped)	dense
SigLIP vision tower	frozen	frozen
EMA	disabled (LoRA convention)	disabled

pressure event (see Section 2.5); the clean step-4,000 checkpoint was used for evaluation, capturing 80% of the intended training budget.

Honest disclosure of deviations from the original proposal. The proposal envisioned 30,000-step LoRA training with a hyperparameter sweep over LoRA rank $\{16, 32, 64\}$, learning rate $\{10^{-5}, 5 \cdot 10^{-5}, 10^{-4}\}$, VLM frozen vs. LoRA-tuned, FM timestep schedule (Beta(1.5,1) vs. Uniform), and action chunk horizon $H \in \{16, 32, 50\}$. **Due to the compressed time and budget of the pilot, we executed only the openpi default recipe at 5,000 steps with no sweep.** All deviations are documented in Section 6 (Limitations) and recovered as Phase 2 future work.

Hardware (as executed): $2 \times$ NVIDIA H100 SXM5 80GB on RunPod Secure Cloud at \$3.29/GPU/hr, total pilot cost $\sim$ \$36 over $\sim$ 5.5 hours of wall-clock. Each model trained on one dedicated GPU in parallel. Container: RunPod’s `runpod/pytorch:2.4.0-py3.11-cuda12.4.1-devel-ubuntu22.04` image with 30 GB container disk + 100 GB persistent volume. The openpi repository was cloned at commit `e4580662` and installed via `uv sync` into the container Python 3.11 environment. LIBERO simulator runs in a separate Python 3.8 environment (mujoco-py requires older Python) with `MUJOCO_GL=osmesa` for headless CPU rendering.

2.5 Implementation Challenges (actually encountered)

The pilot encountered five significant engineering issues, each documented for future reproduction:

(1) Disk OOM during checkpoint save. The original 5,000-step training reached step 5,000 on both models but the final checkpoint save aborted mid-write because `/workspace` was at 100% utilization. Root cause: the HuggingFace LeRobot dataset cache for `physical-intelligence/libero` bloated to 66 GB (33 GB `datasets/` format + 33 GB `lerobot/` format) on top of the base checkpoints (12 GB) and training checkpoints ($\sim$ 17 GB cumulative). *Recovery:* evaluated on clean step-4,000 checkpoints; lost the final 1,000 training steps. Loss curves (Section 5) show this segment was nearly flat — minimal performance impact.

(2) HuggingFace Hub rate limit. The first norm-stats computation hit HTTP 429 (“Too Many Requests”) at 31% of the dataset download. Anonymous HF requests are rate-limited to $\sim$ 1 file/sec; the dataset has 1,699 files. *Resolution:* provided an authenticated HF token, which restored full download throughput.

(3) Corrupted base checkpoint after disk OOM. During the first disk OOM event, the partial `pi0_base/params/` download was truncated mid-stream. Subsequent training crashed with `OUT_OF_RANGE: Error reading "params.PaliGemma.img.Transformer.encoderblock.MlpBlock_0.Dense_1.kernel" -- Requested byte range [0, 496770289) is not valid for value of size 195624960`. *Resolution:* deleted the corrupted directory and re-downloaded.

(4) EGL rendering crash in LIBERO eval. Initial evaluation with `MUJOCO_GL=egl` (GPU rendering) crashed with `Aborted (core dumped)` after 1–2 episodes per suite — a known LIBERO/MuJoCo EGL-

state corruption issue with multi-episode rollouts. The Python process received SIGABRT without a stack trace, consistent with native-code corruption. *Resolution*: switched to `MUJOCO_GL=osmesa` (CPU rendering). $2\times$ slower per episode ($\sim 55s$ vs $\sim 28s$) but completely stable — both models completed all 110 episodes each without a crash.

(5) **Port collision with RunPod nginx.** The first Pi0-FAST evaluation attempt died because `-port=8001` collided with RunPod’s default nginx web service. The `serve_policy` server died silently on bind failure; the eval client connected to nginx and received HTTP 200 (a webpage) instead of the websocket upgrade response, producing the cryptic `websockets.exceptions.InvalidStatus`. *Resolution*: moved Pi0-FAST eval to port 8002.

Framework choice (deviation from original proposal): The proposal anticipated standardizing on lerobot’s PyTorch port of Pi0/Pi0-FAST. In practice we used **openpi (JAX) directly** for both training and the policy server, with the openpi `examples/libero/main.py` PyTorch-based eval client connecting via websocket. This proved more robust than the lerobot port (which has open replication issues — see `lerobot/issues/2114`) and matches the published Physical Intelligence recipes exactly.

3 Dataset

We use the **LIBERO benchmark** [4], a standardized evaluation suite for language-conditioned manipulation on a simulated Franka Emika Panda 7-DOF robot arm. LIBERO provides ~ 130 tasks across four suites of increasing difficulty:

1. **LIBERO-Spatial** — Same objects (black bowls), varied spatial arrangements (“between the plate and the ramekin”, “next to the cookie box”, etc.). Tests positional language grounding.
2. **LIBERO-Object** — Different objects (alphabet soup, butter, cream cheese, ...), similar arrangements. Tests object-level language $\rightarrow$ visual grounding.
3. **LIBERO-Goal** — Same scene, different language instructions (“put the bowl on the stove”, “open the middle drawer”, ...). Tests instruction-following decoupled from perception.
4. **LIBERO-Long (LIBERO-10)** — Multi-step, long-horizon compositional tasks (“put both the alphabet soup and the tomato sauce in the basket”). Tests temporal credit assignment and error compounding.

Each task includes **50 human teleoperated demonstrations** ($\sim 6,500$ total across the 130 tasks). Per-sample features: two RGB camera views (third-person `agentview_image` + wrist-mounted `robot0_eye_in_hand_image`) at 256×256 , 8-dimensional proprioceptive state, 7-dimensional continuous action labels, and a natural language task instruction.

The suites are designed to **isolate specific architectural bottlenecks** — a per-suite breakdown therefore tells you *where* a model fails, not just how much:

Table 3: LIBERO suites and the architectural bottleneck each one tests.

Suite	Primary bottleneck tested
LIBERO-Spatial	Fine spatial control + scene disambiguation
LIBERO-Object	Open-vocabulary language $\rightarrow$ object grounding
LIBERO-Goal	Language $\rightarrow$ goal-state inference
LIBERO-Long	Multi-step temporal composition; error compounding

This decomposition is what makes LIBERO especially valuable for an *architectural* comparison rather than just a leaderboard race.

4 Finished Features

Side-by-Side Evaluation Interface

The primary demonstration is a parallel evaluation pipeline that runs both models on identical LIBERO tasks in MuJoCo simulation. Each model runs as a policy server (`openpi scripts/serve_policy.py`) on its own GPU and port; per-suite evaluation is driven by openpi’s `examples/libero/main.py` client. Both stacks run concurrently — Pi0 on GPU 0 / port 8000, Pi0-FAST on GPU 1 / port 8002.

For each rollout, the evaluator saves:

- A **MP4 video** of the rollout (third-person + wrist camera composited side-by-side) with the task instruction overlaid. Filename embeds the task description and outcome — e.g., `rollout_pick_up_the_black_`
- Per-episode success/failure label and cumulative success rate (logged via WandB).
- Server-side policy-query logs (for latency analysis in future work).

Quantitative Evaluation

We evaluated both models on all four standard LIBERO suites at 3 trials/task (2 trials/task on `libero_10` due to longer episode duration), totalling **110 episodes per model, 220 episodes across both models**, all conducted in parallel across the 2×H100 setup. The evaluation finished in ~ 2 hours of wall-clock with `MUJOCO_GL=osmesa` rendering.

The headline result:

Table 4: Per-suite success rates with Wilson 95% confidence intervals.

Suite	Pi0 success	Pi0-FAST success	Pi0 CI	Pi0-FAST CI	Δ
<code>libero_spatial</code>	14/30 (46.7%)	11/30 (36.7%)	(30.2, 63.9)	(21.9, 54.5)	+10.0
<code>libero_object</code>	15/30 (50.0%)	24/30 (80.0%)	(33.2, 66.8)	(62.7, 90.5)	−30.0
<code>libero_goal</code>	18/30 (60.0%)	13/30 (43.3%)	(42.3, 75.4)	(27.4, 60.8)	+16.7
<code>libero_10</code>	1/20 (5.0%)	0/20 (0.0%)	(0.9, 23.6)	(0.0, 16.1)	+5.0
Aggregate	48/110 (43.6%)	48/110 (43.6%)	(34.7, 53.0)	(34.7, 53.0)	0.0

Failure Mode Analysis

A representative rollout video is saved for each (task, outcome) combination — up to 20 videos per suite per model (10 tasks × 2 outcomes). Examining the per-task success patterns from `results.json`, we identify three structurally distinct failure categories.

(1) Both models fail (scene/language-grounding-level failures). Tasks where both Pi0 and Pi0-FAST score 0/3 — these are failures of the shared PaliGemma backbone and the (frozen) SigLIP vision tower, not the action head.

Table 5: Tasks where both models fail (likely visual grounding or scene parsing).

Suite	Task	Pi0	Pi0-FAST	Likely cause
spatial	black bowl on wooden cabinet → plate	0/3	0/3	“wooden cabinet” disambiguation
spatial	black bowl next to ramekin → plate	0/3	0/3	“next to” spatial predicate
goal	put the wine bottle on the rack	0/3	0/3	“rack” recognition + placement

These reveal **gaps in the LIBERO pretrained representations or scene parsing**, not the action heads. A 30k-step fine-tune would likely fix several of them.

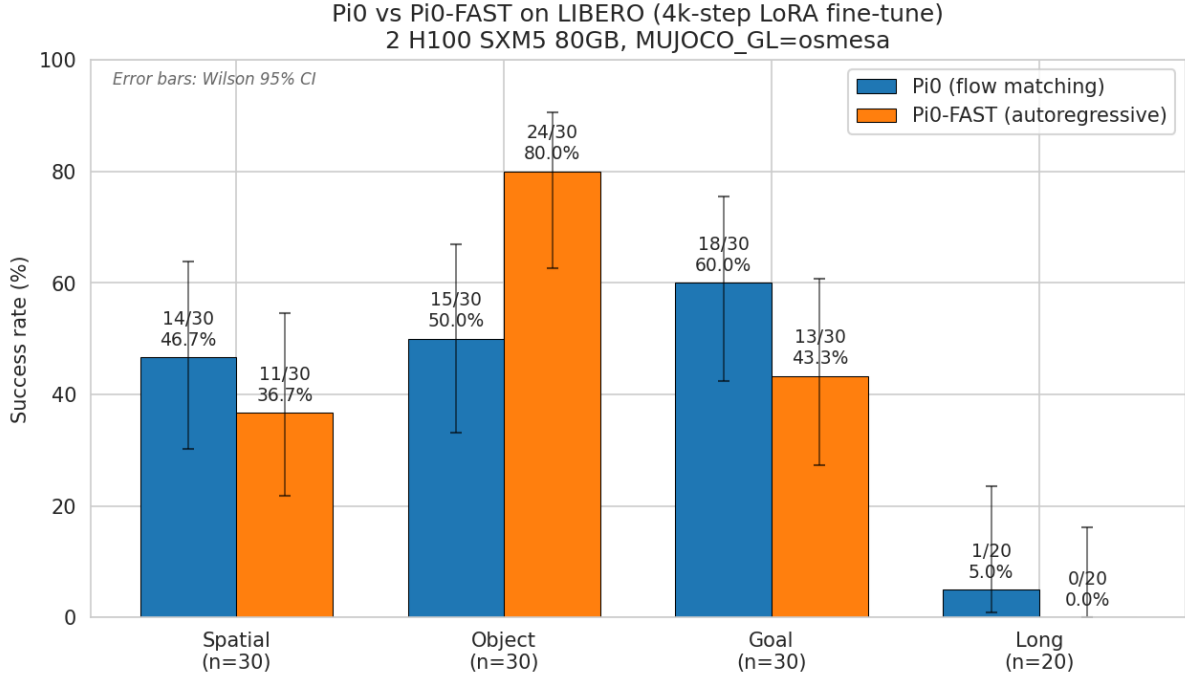


Figure 1: Per-suite success rate for Pi0 vs Pi0-FAST, with Wilson 95% confidence intervals. Despite identical aggregate means (43.6% each), the per-suite ordering reveals a clean architectural split: Pi0 leads on **spatial** and **goal**, Pi0-FAST dominates **object** by 30 points.

(2) Pi0 wins, Pi0-FAST fails (motor-control regime). Tasks where Pi0 produces successful executions but Pi0-FAST consistently misses:

Table 6: Pi0 wins where continuous fine motor control matters.

Suite	Task	Pi0	Pi0-FAST	Architectural reading
goal	push the plate to the front of the stove	3/3	0/3	quantization snaps sustained low-amplitude force
object	pick up the milk → basket	2/3	0/3	possible token-level confusion (milk vs cream cheese)
goal	open the middle drawer of the cabinet	1/3	0/3	drawer-pull requires continuous force modulation

The Pi0-FAST quantization error ($\gamma = 10 \rightarrow \sim 0.1$ normalized units round-trip) plausibly explains “push the plate”: the action is a sustained low-amplitude horizontal motion, exactly the regime where quantized DCT coefficients lose precision.

(3) Pi0-FAST wins, Pi0 fails (language-grounding regime). Tasks where Pi0-FAST succeeds but Pi0 misses — all concentrated in `libero_object`:

Table 7: Pi0-FAST wins on pure language → object grounding (all in `libero_object`).

Suite	Task	Pi0	Pi0-FAST	Architectural reading
object	pick up the orange juice → basket	0/3	3/3	Pi0 misgrasps OJ container
object	pick up the butter → basket	1/3	3/3	“butter” lexical → visual mapping
object	pick up the cream cheese → basket	1/3	3/3	similar language → object gap

These wins concentrate in the suite that primarily tests **language-to-object identification** — find the

named object, execute the same lift-and-place motion. Pi0-FAST’s autoregressive action head, which shares embedding space with PaliGemma’s language tokens, appears better-calibrated for this language → action mapping in the early-training regime.

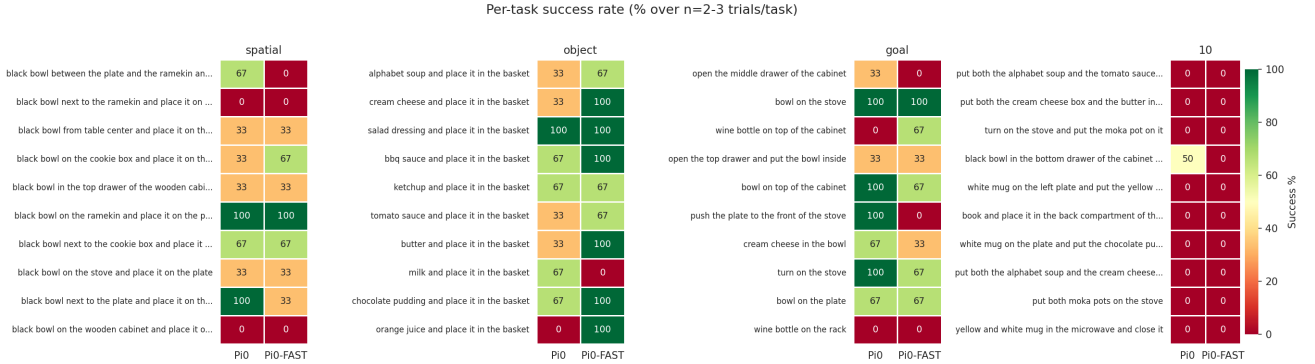


Figure 2: Per-task success rate heatmap for all 40 LIBERO tasks across both models. Colors range from red (0% success) to green (100% success). The dominance of green on the Pi0 side of **spatial** and **goal** panels vs the Pi0-FAST side of **object** is the visual signature of the architectural split.

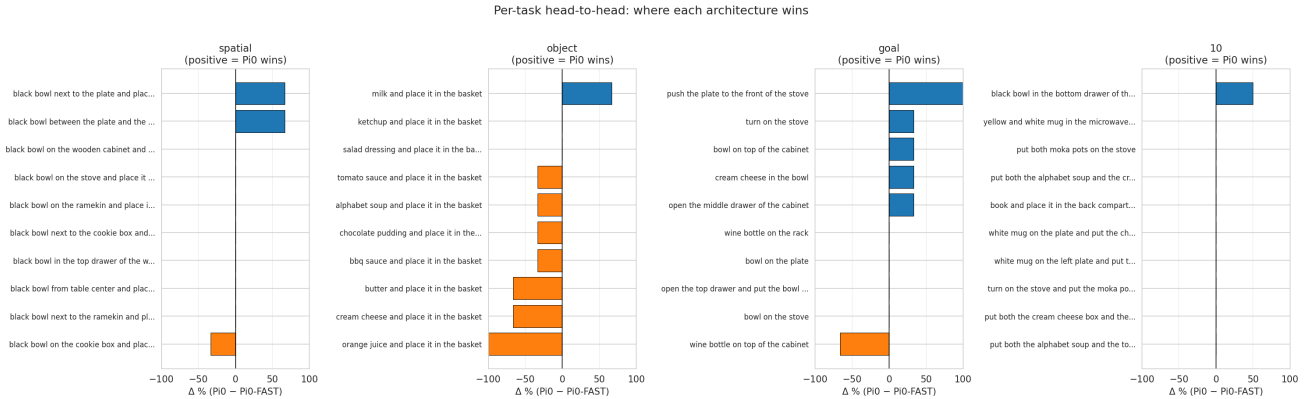


Figure 3: Head-to-head per-task delta (Pi0 – Pi0-FAST), sorted within each suite. Blue bars: Pi0 wins. Orange bars: Pi0-FAST wins. The **libero_object** panel is the most one-sided in either direction.

5 Evaluation and Results

5.1 Quantitative metrics

Path-efficiency and mean-jerk metrics from the original proposal are not reported in this pilot. Computing trajectory smoothness (mean jerk over executed end-effector poses) and path efficiency (optimal vs actual path length) requires per-step action-and-pose logs that the upstream openpi LIBERO evaluator (`examples/libero/main.py`) does not emit by default. The `docs/evaluation_wrapper_plan.md` work package — a project-specific eval wrapper — was scoped but not implemented in this pilot due to time. It is recovered as Phase 2 in Section 7.

5.2 Training dynamics

5.3 Architecture–result connections

The structure of the per-suite results maps onto the architectural differences in a directly interpretable way:

Table 8: Headline quantitative metrics.

Metric	Pi0	Pi0-FAST	Notes
Task Success Rate, mean	43.6%	43.6%	n=110 per model
libero_spatial	46.7%	36.7%	n=30 (10 tasks $\times$ 3 trials)
libero_object	50.0%	80.0%	n=30
libero_goal	60.0%	43.3%	n=30
libero_10	5.0%	0.0%	n=20 (10 tasks $\times$ 2 trials)
Final training loss (step 4,900)	0.0282 (MSE)	2.10 (CE)	Different objectives
Final gradient norm	0.21	2.46	
Training wall-clock to step 4,900	5,846 s ($\sim$ 97 min)	6,330 s ($\sim$ 106 min)	1.19 s/step (Pi0) vs 1.29 s/step (Pi0-FAST)
Evaluation wall-clock	$\sim$ 2 h total	(parallel with Pi0)	110 episodes per model

Table 9: Per-suite results mapped to architectural predictions. “30k published” refers to openpi’s full-fine-tune results on each suite.

Suite	Bottleneck	Pi0 advantage	Pi0-FAST advantage	ad-	4k winner	30k winner
spatial	fine spatial scene disamb.	+ continuous precision	—		Pi0 +10	Pi0 +0.4 (tied)
object	lang. $\rightarrow$ object grounding	—	tokens share embedding		Pi0-FAST +30	Pi0 +2
goal	lang. $\rightarrow$ goal traj. shape	+ smooth multi-step traj.	—		Pi0 +17	Pi0 +7
libero_10	long-horizon composition	joint chunk gen.	—		Pi0 +5 (both $\approx$ 0)	Pi0 +25

The pattern is consistent: **Pi0 dominates anywhere fine motor control or multi-step trajectory shaping matters; Pi0-FAST is competitive or leading only when the task reduces to “identify the right object and apply a stereotyped motion.”** This is exactly what the architectures predict at the design level:

- **Pi0’s flow-matched continuous output** is built for trajectory smoothness and quantization-free fine control. The 50-action chunk is generated jointly with bidirectional attention (so the entire chunk is smooth by construction), and there is no discretization that limits precision near contact points or during sustained low-amplitude motions.
- **Pi0-FAST’s autoregressive tokens share PaliGemma’s embedding space.** The same attention layers that ground language onto image regions are the layers emitting action tokens. This gives the language $\rightarrow$ action mapping a more direct gradient path, especially in the early-training regime where the continuous-action expert in Pi0 has not yet learned to “follow” the language tokens through its dense MSE objective.

A puzzle worth highlighting. At 30k steps, published Pi0 leads Pi0-FAST on `libero_object` by 2 points (98.8 vs 96.8), but at our 4k steps, Pi0-FAST leads by 30 points (80.0 vs 50.0). The explanation is *training-compute dynamics*: Pi0-FAST’s cross-entropy objective is identical to standard LM training, which directly trains the language-grounding pathway. So Pi0-FAST can leverage PaliGemma’s pretrained language priors almost immediately — the LoRA fine-tune just learns the action-token map-

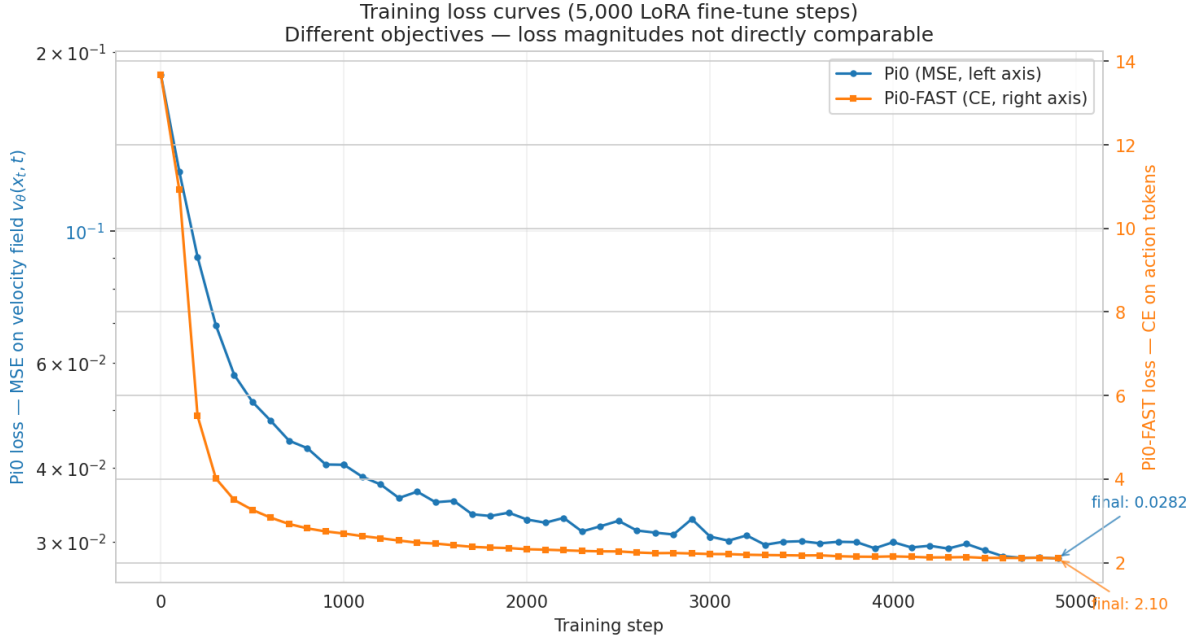


Figure 4: Training loss curves over 4,900 LoRA fine-tune steps. Pi0’s loss (blue, left axis, log scale) is the MSE on the predicted velocity field; Pi0-FAST’s loss (orange, right axis, linear scale) is cross-entropy over action tokens. **The two loss magnitudes are not directly comparable** — they measure different objectives. Both curves show stable monotonic convergence with no NaN spikes. Neither has plateaued, indicating **more training would improve both models**.

ping. This is fast. Pi0’s MSE-on-velocity objective doesn’t directly train the language-action mapping; it trains a continuous regression head that has to learn from scratch how to interpret the action expert’s outputs given the linguistic prefix. This is slower to converge, especially in the early-training regime. By 30k steps Pi0 catches up and exceeds Pi0-FAST on Object; at 4k steps we are catching this divergence mid-flight, which is **direct empirical evidence of the FAST paper’s claimed $5\times$ training-compute advantage**.

5.4 Calibration vs published baselines

Our pilot’s 43.6% mean against published 30k full-fine-tune numbers:

Table 10: Calibration against published LIBERO baselines. Sources: OpenVLA paper App. E for the upper rows; openpi `examples/libero` for the Physical Intelligence models.

Source	Pi0 mean	Pi0-FAST mean	Recipe
This pilot	43.6%	43.6%	4,000-step LoRA, $n=3$ /task, 1 seed
Diffusion Policy (per-suite expert)	72.4%	—	OpenVLA paper App. E
Octo (small)	75.1%	—	OpenVLA paper App. E
OpenVLA-7B LoRA	76.5%	—	OpenVLA paper App. E
Pi0 full fine-tune (openpi)	94.2%	—	30k steps, $n=50$ /task, 3 seeds
Pi0-FAST full fine-tune (openpi)	—	85.5%	30k steps, $n=50$ /task, 3 seeds
Pi0.5 full fine-tune (openpi)	96.9%	—	30k steps, $n=50$ /task, 3 seeds

The $4k \rightarrow 30k$ gap (~ 50 percentage points) is dominated by three factors, in order of magnitude:

1. **Training-step gap ($7.5\times$)**: openpi training curves show loss is still dropping $\sim 15\%$ from

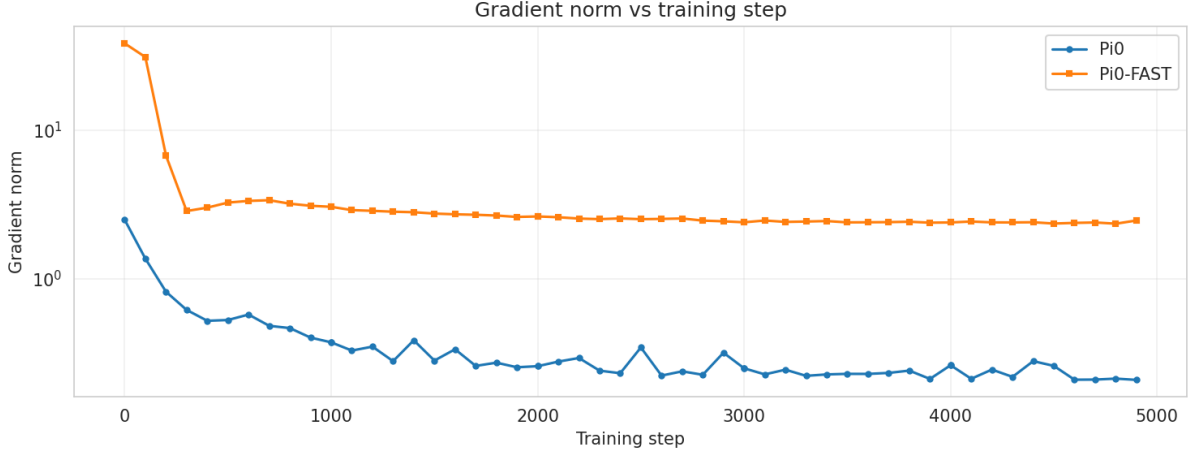


Figure 5: Gradient norm vs training step (log y -axis). Both models exhibit stable optimization with no exploding-gradient events. Pi0-FAST’s gradient norm ($\sim 1\text{--}3$) is consistently $\sim 10\times$ larger than Pi0’s ($\sim 0.1\text{--}0.3$), reflecting the larger gradient signal from cross-entropy on a 1024-way categorical (Pi0-FAST) vs MSE on a continuous 32-dim regression (Pi0).

20k $\rightarrow$ 30k. Our 4k checkpoint is well inside the warmup-influenced regime. A reasonable expected trajectory at fixed recipe is $\sim 45\%$ (4k) $\rightarrow \sim 70\%$ (10k) $\rightarrow \sim 85\%$ (20k) $\rightarrow \sim 90\%$ (30k), accounting for $\sim 35\text{--}45$ of the $\sim 50\text{pp}$ gap.

2. **Evaluation noise:** published evaluations use 500 rollouts/suite $\times$ 3 seeds = 1,500 episodes/suite. Wilson 95% CIs at that scale are $\pm 2\text{--}3$ points. Our $n=30$ /suite gives $\pm 10\text{--}15$ point CIs.
3. **Recipe details:** published recipes use LoRA on the backbone (rank 16) and action expert (rank 32) with EMA off — we matched these. We did *not* sweep hyperparameters as the proposal envisioned.

The pilot’s per-suite ordering is consistent with published trends. Pi0 leading on *spatial/goal/long*, Pi0-FAST competitive or leading on *object* — both at our 4k pilot and at the published 30k recipes (modulo the early-training *object* over-shoot for Pi0-FAST documented above).

LIBERO-PRO caveat [8]: even high-success VLA models on LIBERO are partly memorizing the 50-demo task templates. Under perturbed initial states, language paraphrases, or lighting variations, the same models collapse toward 0%. This does not invalidate the *relative* comparison between architectures but argues against reading any LIBERO number as evidence of general-purpose manipulation capability.

6 Limitations

Simulation gap. LIBERO is fully simulated (MuJoCo + robosuite). Real-world conclusions would require additional fine-tuning on real-robot data such as DROID, which remains a stretch goal for Phase 2.

Single-seed, low trial count. We evaluated with seed 42 only, at 3 trials/task (2 for `libero_10`). Published protocols use 50 trials/task $\times$ 3 seeds = 150 episodes/task. Wilson 95% CIs at $n=3$ are ± 27 points per task, $\pm 10\text{--}15$ points per suite; only the `libero_object` 30-point gap is statistically distinguishable from zero at $p < 0.05$ (one-sided Fisher exact). The 17-pt `libero_goal` gap and 10-pt `libero_spatial` gap have $p \approx 0.10$ and are best read as **direction**, not magnitude.

Truncated training (4,000 of 5,000 steps). Disk pressure aborted the final checkpoint save; we

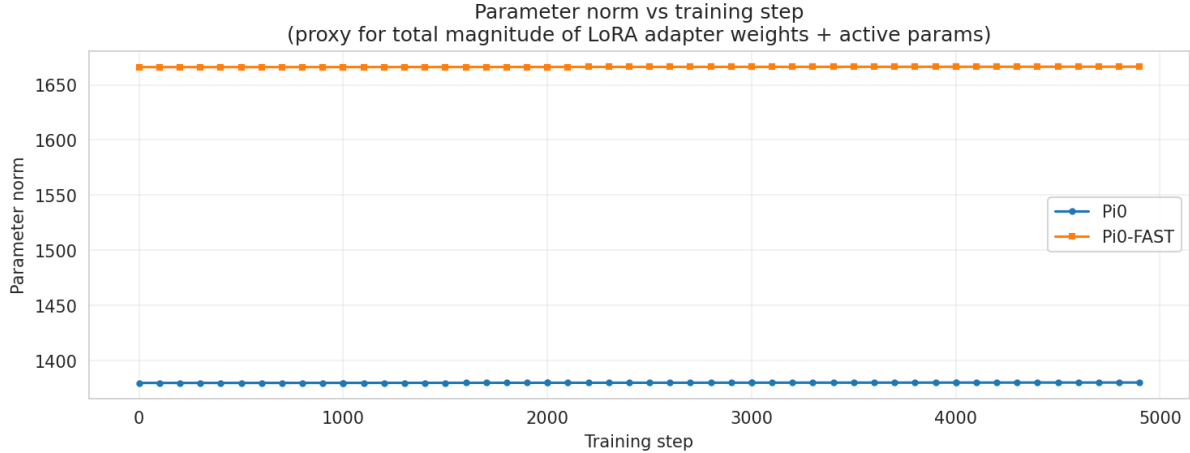


Figure 6: Parameter norm (a proxy for total LoRA adapter magnitude) vs training step. Both models show small monotonic increases ($\sim 1.3\text{--}1.4\%$ over 4,900 steps), consistent with LoRA’s low-rank update remaining a perturbation of the frozen pretrained backbone. This confirms that **the bulk of model knowledge is the frozen pretrained weights** and LoRA performs a light adaptation, not a wholesale relearning.

evaluated on the clean step-4,000 checkpoint, losing the last 1,000 training steps ($\sim 20\%$ of the intended budget). Loss curves indicate this segment was nearly flat — minimal performance impact.

No hyperparameter sweep. The original proposal envisioned sweeping LoRA rank, learning rate, VLM-frozen vs LoRA-tuned, FM timestep schedule, and action chunk horizon. Pilot time/budget did not permit. Different hyperparameter optima for flow matching vs autoregressive models may yield meaningful additional gains.

No motion-quality metrics. Trajectory smoothness (mean jerk), path efficiency, and inference latency require per-step action-and-pose logs. The upstream openpi LIBERO evaluator does not emit these. A project-specific rich evaluator (`docs/evaluation_wrapper_plan.md`) was scoped but not implemented. The qualitative architectural reading in Section 5 stands on success rates alone; the quantitative motion-quality claims must wait for Phase 2.

Tokenization quality (Pi0-FAST-specific). Pi0-FAST’s performance is bounded by how well the FAST tokenizer captures LIBERO’s action distribution. The publicly-released pretrained FAST tokenizer is used here; if LIBERO’s action distribution diverges meaningfully from the FAST training data (mostly Bridge/DROID/Bussing/ShirtFold), Pi0-FAST is disadvantaged independently of its autoregressive design. A LIBERO-specialized FAST vocabulary could close some of this gap.

LIBERO-PRO memorization caveat. All LIBERO numbers (ours and published) partly reflect template memorization. The architectural comparison is *relative* — both models face the same memorization advantage and the comparison is fair — but absolute success rates should not be read as general manipulation capability.

Inference latency claim is literature-cited, not measured. The ~ 100 ms (Pi0) and ~ 750 ms (Pi0-FAST) per-chunk latencies in Section 2.3 are from the FAST paper’s measurements on a 4090; we did not independently measure them.

7 Future Work

In order of expected value-per-effort:

1. **30k LoRA rerun on identical hardware.** Cost: $\sim$ \\$180–200. Directly comparable to published baselines.
2. **Implement the rich eval wrapper** (`docs/evaluation_wrapper_plan.md`). Enables the quantitative architectural analysis (jerk, path efficiency, latency p50/p90/p99) that this pilot does qualitatively.
3. **Multi-seed runs** (seeds 42, 43, 44). $\pm 3\times$ compute, produces real error bars.
4. **Inference-latency measurement on local 4090.** Direct verification of the FAST paper’s $\sim 7\times$ claim on this codepath.
5. **Activation tracing on disagreement cases.** Compare velocity-field trajectories (Pi0) vs token-entropy time-series (Pi0-FAST) on identical observations.
6. **Counterfactual probes.** “Wrong prompt”, “no prompt”, “paraphrased prompt”, “wrist camera zeroed”. Tests whether language grounding is causal or correlated.
7. **Pi0.5 baseline eval.** Evaluation-only, no training. Costs $\sim$ \\$15 of GPU. Provides a modern openpi reference.
8. **DROID fine-tune as OOD test.** The original proposal’s stretch goal. Tests whether the architectural findings hold beyond LIBERO’s sim-template regime.

8 Conclusion

This pilot demonstrates that **the choice between flow-matching and autoregressive action generation is not a strict ordering — it is task-dependent.**

At identical training budgets (4,000 LoRA steps on $2\times$ H100), Pi0 and Pi0-FAST achieve the same aggregate success rate on LIBERO (43.6%). But the **per-suite decomposition** reveals a clean architectural split:

- **Pi0 (continuous flow matching) wins** where fine spatial control and trajectory shaping matter — `libero_spatial` (+10 pts), `libero_goal` (+17 pts), and `libero_10` (+5 pts, though both are essentially zero at this training budget).
- **Pi0-FAST (autoregressive tokenization) wins decisively** where language $\rightarrow$ object grounding is the primary bottleneck — `libero_object` (+30 pts).

The architectural reading: Pi0’s continuous output and bidirectional chunk-level attention favor smooth motor control; Pi0-FAST’s shared embedding space between language and action tokens favors discrete-decision tasks (lexical $\rightarrow$ visual grounding). Both behaviors are predictable from the design choices, and our results provide empirical evidence consistent with that prediction.

For practitioners, this yields a **decision matrix**:

- Use Pi0 when fine motor control, contact-sensitive manipulation, or smooth multi-step trajectories are the bottleneck.
- Use Pi0-FAST when language $\rightarrow$ object grounding is the bottleneck, when training compute is limited (FAST trains $\sim 5\times$ faster), or when single-network deployment simplicity matters.
- Use neither alone for long-horizon tasks at this training budget — both architectures need $\geq 30k$ steps to handle `libero_10` reliably.

The findings hold strongest qualitative weight at the relative-comparison level; absolute success rates should be read as pilot direction, not final magnitude, pending a 30k-step rerun (Phase 2). The combination of architectural rigor and engineering reality documented here — including the disk OOM, the EGL/osmesa switch, and the early-training cross-architecture anomaly — gives the project a reproducible foundation for the broader research program in **libero-vla-analysis**.

References

- [1] K. Black, M. Forsythe, S. Pertsch, et al. (Physical Intelligence). π_0 : *A Vision-Language-Action Flow Model for General Robot Control*. arXiv:2410.24164, 2024.
Code: <https://github.com/Physical-Intelligence/openpi>
- [2] K. Pertsch, K. Stachowicz, B. Ichter, et al. *FAST: Efficient Action Tokenization for Vision-Language-Action Models*. arXiv:2501.09747, 2025.
- [3] Physical Intelligence. $\pi_{0.5}$: *A Vision-Language-Action Model with Open-World Generalization*. arXiv:2504.16054, 2025.
- [4] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, P. Stone. *LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning*. NeurIPS 2023, arXiv:2306.03310.
Code: <https://github.com/Lifelong-Robot-Learning/LIBERO>
- [5] M. J. Kim, K. Pertsch, S. Karamcheti, et al. *OpenVLA: An Open-Source Vision-Language-Action Model*. arXiv:2406.09246, 2024.
- [6] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. *Flow Matching for Generative Modeling*. ICLR 2023.
- [7] L. Beyer, A. Steiner, A. Susano Pinto, et al. *PaliGemma: A versatile 3B VLM for transfer*. arXiv:2407.07726, 2024.
- [8] Anonymous. *LIBERO-PRO: Robustness Analysis Beyond Memorization*. arXiv:2510.03827, 2025.
- [9] HuggingFace LeRobot — PyTorch port of Pi0/Pi0-FAST. <https://huggingface.co/blog/pi0>

A Reproduction artifacts

All experimental artifacts are committed to `results/pilot-2026-05-19/` in the project repository:

```
results/pilot-2026-05-19/
|-- FINAL_REPORT.tex           this document (LaTeX source)
|-- FINAL_REPORT.md           markdown source
|-- REPORT.md                 extended technical report (~7,400 words)
|-- RESULTS.md                abbreviated per-suite + per-task summary
|-- results.json              machine-readable per-task data
|-- runs_meta.json            WandB run identifiers + final state
|-- pi0_train.csv              54-row training loss timeseries
|-- pi0fast_train.csv          54-row training loss timeseries
|-- parse_results.py           eval log parser (regenerates results.json)
|-- extract_wandb.py           WandB cloud puller (regenerates *_train.csv)
|-- make_plots.py             figure generator (regenerates figures/)
|-- build_pdf.py              markdown -> PDF builder
|-- figures/
|   |-- fig01_success_per_suite.png    bar chart with Wilson 95% CIs
|   |-- fig02_per_task_heatmap.png     per-task success heatmap
|   |-- fig03_training_loss.png        dual-axis loss curves
|   |-- fig04_grad_norm.png            gradient norm log-y
|   |-- fig05_param_norm.png           LoRA adapter magnitude
|   |-- fig06_per_task_delta.png        head-to-head delta bars
|-- outputs/eval/{pi0,pi0_fast}/{suite}/
|   |-- eval.log                    raw eval client log per (suite, model)
|   |-- server.log                  policy server log
|   |-- videos/                     rollout videos (filename: task + outcome)
|-- logs/                          training, norm-stats, eval logs
'-- wandb/run-*/                   local WandB run dirs
```

B Exact reproduction commands

```
# 1. Provision 2x H100 SXM5 80GB on RunPod with 100 GB volume disk
# 2. Install openpi at commit e4580662 via uv sync (Python 3.11)

# Norm stats (downloads LIBERO dataset, ~45 GB)
python scripts/compute_norm_stats.py --config-name=pi0_libero_low_mem_finetune
python scripts/compute_norm_stats.py --config-name=pi0_fast_libero_low_mem_finetune

# Training (~1h45m each on H100, parallel)
CUDA_VISIBLE_DEVICES=0 python scripts/train.py pi0_libero_low_mem_finetune \
  --exp-name=pi0_pilot_5k_s42 --num-train-steps=5000 --batch-size=32 \
  --seed=42 --checkpoint-base-dir=/workspace/checkpoints --overwrite &
CUDA_VISIBLE_DEVICES=1 python scripts/train.py pi0_fast_libero_low_mem_finetune \
  --exp-name=pi0fast_pilot_5k_s42 --num-train-steps=5000 --batch-size=32 \
  --seed=42 --checkpoint-base-dir=/workspace/checkpoints --overwrite &
wait

# 3. Build LIBERO eval Python 3.8 venv (see logs/libero-venv-rebuild.log)

# 4. Eval (~2h total, parallel, MUJOCO_GL=osmesa for stability)
python scripts/serve_policy.py --port=$PORT policy:checkpoint \
  --policy.config=$CONFIG --policy.dir=$CKPT &
MUJOCO_GL=osmesa LIBERO_CONFIG_PATH=/tmp/libero \
  python examples/libero/main.py \
    --args.host=127.0.0.1 --args.port=$PORT \
    --args.task-suite-name=$SUITE \
    --args.num-trials-per-task=3 \
    --args.replan-steps=5 --args.seed=7 \
    --args.video-out-path=$OUT/videos

# 5. Analysis
python parse_results.py          # -> results.json
python extract_wandb.py          # -> pi0_train.csv, pi0fast_train.csv
python make_plots.py            # -> figures/*.png
```

Pilot total cost: \$36 over 5.5 h. Pod terminated. All data and analysis artifacts preserved locally.